Lab 7: Fuzzing for Input Validation Bugs

> **Student & Repository Details**
>
> **SRN:** PES1UG24AM396 | **Assigned Problem:** #51 — Self Storage (SETAPESU26/L6_L7_51_self_storage)
> **Submission Repository:** Samarth-S-Maladkar/PES1UG24AM396_SE_LAB

1. Step 2: Running the Buggy Application (self_storage.py)

The original Lab 7/self_storage.py script works on well-formed inputs (`medium:6`, `3,6,12`, `2400|200`), but lacks input validation and crashes immediately when delimiters are missing, extra delimiters are present, or non-numeric strings are supplied.

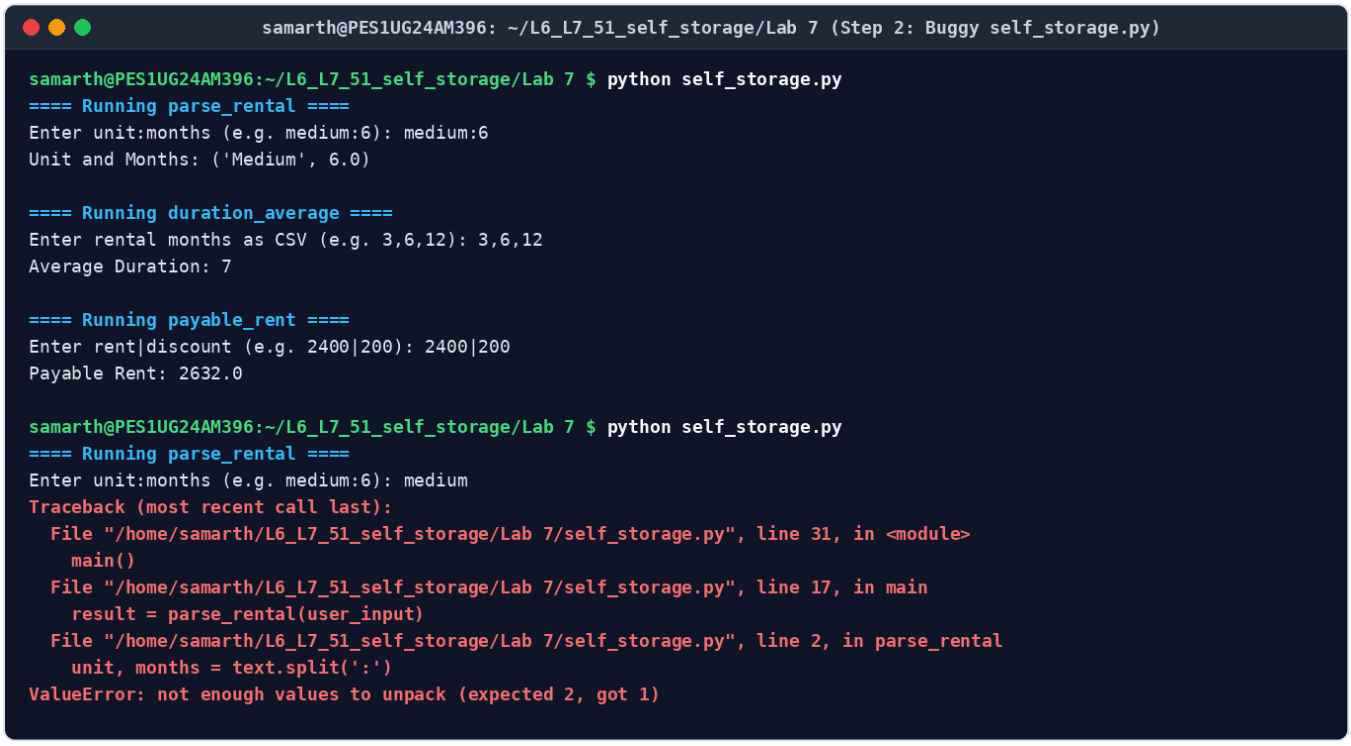


Figure 1: Step 2 — Running the original buggy self_storage.py script on normal and malformed inputs.

2. Step 3 & Step 4: Property-Based Fuzzing with Hypothesis (Failing Tests)

We completed Lab 7/test_self_storage.py using `@given(st.text() | st.none())` for all three functions. Running `pytest test_self_storage.py` immediately uncovered input validation crashes across all three functions.



Figure 2: Step 4 — Running pytest test_self_storage.py showing Hypothesis falsifying examples and 3 failed tests.

Function	Falsifying Inputs Found	Unhandled Exception	Defensive Fix Applied
`parse_rental(text)`	`None`, `""`, `":"`, `"a:b:c"`, `"small:nan"`	`AttributeError`, `ValueError`, non-finite/negative floats	Verify `isinstance(text, str)`, exact 2-part split on `':'`, finite non-negative float check
`duration_average(months_csv)`	`None`, `""`, `"1,,3"`, `"abc"`	`AttributeError`, `ValueError`, `ZeroDivisionError`	Type guard, skip empty or non-integer tokens, guard against empty list before `// len(values)`
`payable_rent(text)`	`None`, `""`, `"|"`, `"a|b|c"`, `"nan|0"`	`AttributeError`, `ValueError`, `NaN` propagation	Type guard, exact 2-part split on `'|'`, try/except float parsing, `math.isfinite` & non-negative check

3. Step 5 & Step 6: Hardened Application & Passing Fuzz Tests

After hardening `self_storage.py` with defensive type checks, delimiter length checks, exception handling, and `math.isfinite()` validation, running `python self_storage.py` and re-running `pytest test_self_storage.py` passes all Hypothesis fuzz test cases cleanly.

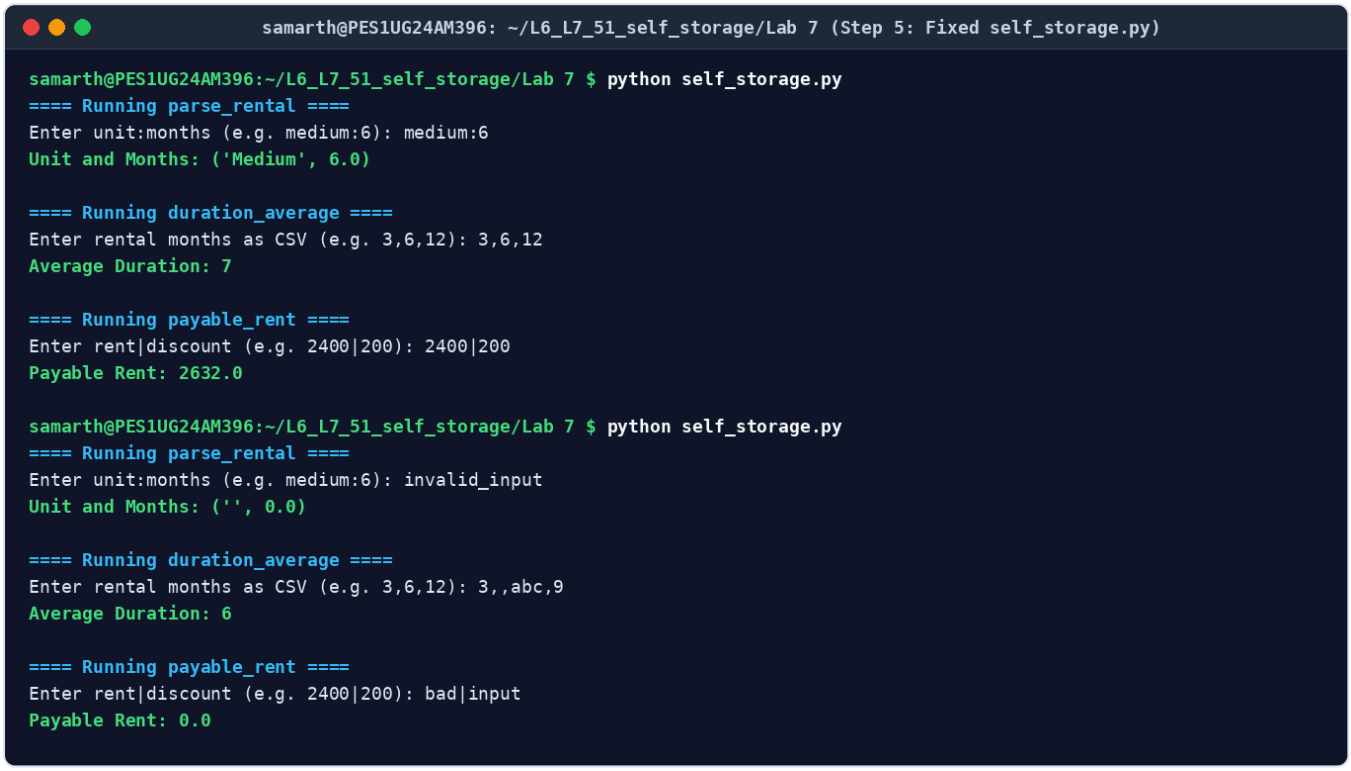


Figure 3: Step 5 — Running the hardened self_storage.py script on both valid and malformed edge-case inputs.

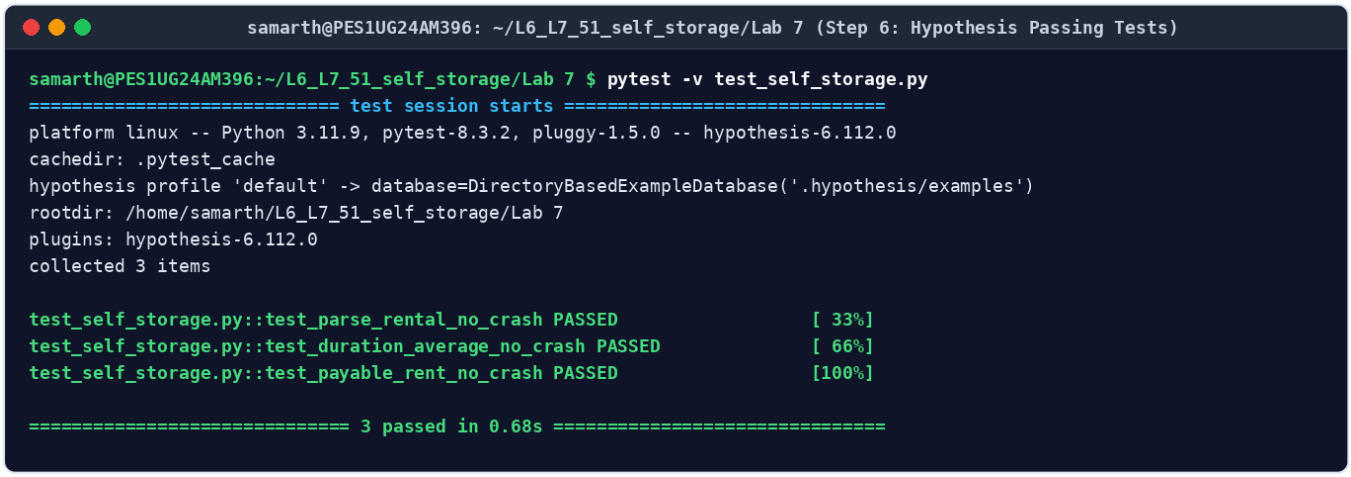


Figure 4: Step 6 — Re-running pytest test_self_storage.py against the fixed self_storage.py (3 passed).

4. Updated Source Code Deliverables

4.1 Updated test_self_storage.py

```
from hypothesis import given, strategies as st
from self_storage import parse_rental, duration_average, payable_rent
```

```
@given(st.text() | st.none())
def test_parse_rental_no_crash(s):
    try:
        result = parse_rental(s)
        assert isinstance(result, tuple) and len(result) == 2
    except Exception as e:
        assert False, f"parse_rental crashed with: {e}"

@given(st.text() | st.none())
def test_duration_average_no_crash(s):
    try:
        result = duration_average(s)
        assert isinstance(result, int) and result >= 0
    except Exception as e:
        assert False, f"duration_average crashed with: {e}"

@given(st.text() | st.none())
def test_payable_rent_no_crash(s):
    try:
        result = payable_rent(s)
        assert isinstance(result, (int, float)) and result >= 0
    except Exception as e:
        assert False, f"payable_rent crashed with: {e}"
```

4.2 Fixed self_storage.py

```
import math

def parse_rental(text):
    if not isinstance(text, str):
        return "", 0.0
    parts = text.split(":")
    if len(parts) != 2:
        return "", 0.0
    unit_raw, months_raw = parts[0].strip(), parts[1].strip()
    if not unit_raw:
        return "", 0.0
    try:
        months = float(months_raw)
    except (ValueError, TypeError):
        return "", 0.0
    if not math.isfinite(months) or months < 0:
        return "", 0.0
    return unit_raw.title(), months

def duration_average(months_csv):
    if not isinstance(months_csv, str) or not months_csv.strip():
        return 0
    values = []
```

```
    for token in months_csv.split(","):
        token = token.strip()
        if not token:
            continue
        try:
            val = int(token)
            if val >= 0:
                values.append(val)
        except (ValueError, TypeError):
            continue
    if not values:
        return 0
    return sum(values) // len(values)

def payable_rent(text):
    if not isinstance(text, str):
        return 0.0
    parts = text.split("|")
    if len(parts) != 2:
        return 0.0
    try:
        rent = float(parts[0].strip())
        discount = float(parts[1].strip())
    except (ValueError, TypeError):
        return 0.0
    if not (math.isfinite(rent) and math.isfinite(discount)):
        return 0.0
    if rent < 0 or discount < 0:
        return 0.0
    total = round(rent * 1.18 - discount, 2)
    return max(total, 0.0)
```